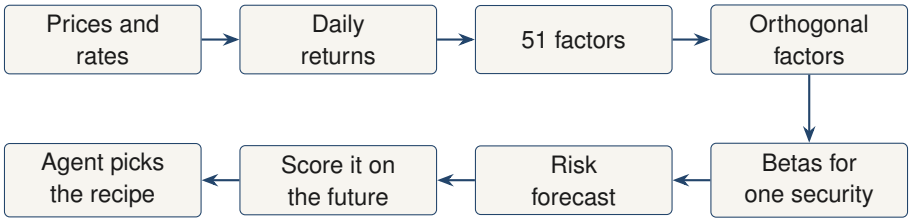
The Factor Model Cookbook

How the daily multi-asset factor model is made,
step by step, as if you did it by hand



Companion to `factor-model-paper.pdf` and `model-agent.md`.
Every rule here is the rule the code applies; the file that applies it
is named at the end of each recipe.

October 2026

Contents

Before you start **3**

1 Preparing the ingredients **4**

1.1 Stock the pantry . . . 4
1.2 Lay out the calendar . . . 5
1.3 Turn prices into daily returns . . . 5
1.4 Subtract the cash rate . . . 5
1.5 Put everything in dollars . . . 6

2 Cooking the factors **6**

2.1 The single-fund factor . . . 7
2.2 The long-short spread . . . 7
2.3 The basket . . . 7
2.4 A bond return from a yield . . . 7
2.5 Yield-curve portfolios: level, slope, curvature . . . 8
2.6 Currency factors . . . 9
2.7 Factors from a rate or an index level: standardised changes . . . 9
2.8 Volatility factors . . . 10
2.9 Momentum strategies . . . 11
2.10 Sector factors . . . 11
2.11 The full menu . . . 11

3 Separating the flavours **13**

3.1 Orthogonalise down the hierarchy . . . 13
3.2 Check that the style factors are what they say . . . 14
3.3 Taste every factor: the stationarity battery . . . 14

4 Fitting one security **15**

4.1 Prepare the security . . . 15
4.2 Choose the factors . . . 16
4.3 Run the rolling regression . . . 16

5 Forecasting the risk **17**

5.1 Estimate the factor covariance . . . 17
5.2 Systematic risk, with the noise taken out . . . 18

5.3 Specific risk 18
5.4 Put the forecast together 18
5.5 The one-month forecast 19

6 Tasting the dish: scoring the forecast 19

7 Choosing the recipe: the model agent 20
7.1 Lock away the test data 20
7.2 Gather the candidates 21
7.3 Pick the winner 21
7.4 Open the drawer once: the verdict 22

8 Serving a portfolio 22

A Letting the machine cook 23

B Kitchen glossary 24

Before you start: how to read this book

This book explains the whole model as a set of recipes. Each recipe says what ingredients you need, what to do with them in order, and how to check the result. Nothing in it needs a computer in principle: with a spreadsheet, a calculator and a lot of patience you could rebuild every number the terminal shows. In practice the code does it, and the last line of each recipe names the file that does.

The model answers one question:

> *How much could this security's value move tomorrow, and over the next month, and which kinds of market risk is that movement made of?*

It does this without looking inside the security. It never asks what a fund owns. It only watches how the security's daily return moves together with a set of **factors**: daily returns that each stand for one kind of market risk, such as "global stocks", "US interest rates", "the dollar" or "oil". If a security has gone up by about 1.2% on days global stocks went up 1%, it has a *loading* (a *beta*) of about 1.2 on global stocks.

The work happens in seven stages, which are the seven parts of this book:

1. **Prepare the ingredients**: turn raw prices and rates into clean daily returns on one calendar, in dollars, above cash.
2. **Cook the factors**: build 40 core factors and 11 sector factors from those returns.
3. **Separate the flavours**: strip from each factor what the factors above it already explain (orthogonalisation), and test that every factor is a well-behaved series.
4. **Fit one security**: measure its betas on the factors, over a rolling window.
5. **Forecast its risk**: combine the betas with the factors' own risk, add the part the factors cannot explain.
6. **Taste the dish**: compare the forecast with what actually happened next.
7. **Choose the recipe**: let the model agent try many variants and pick the simplest one that is as good as the best, then judge it once on data it never saw.

A last part shows how the same forecast is applied to a portfolio.

Why it is done this way

Two rules run through every recipe. **Never use tomorrow's information today**: every number dated t is computed only from data known by the close of day t. And **never invent data**: a value that is missing stays missing; it is not filled in from the day before unless the gap is a short holiday.

Notation used throughout

P_t	closing price on day t (adjusted for dividends and splits)
r_t	daily log return, $\ln(P_t / P_{t-1})$
c_t	cash rate on day t, the Federal Funds effective rate (FRED `DFF`), in percent per year
f_t	a factor's return on day t
β	a loading: how much the security moves per unit move of a factor
Σ	the covariance matrix of the factors: how much each moves, and how they move together
σ	a volatility (standard deviation), quoted per year unless stated
252	trading days in a year; a daily number times $\sqrt{252} \approx 15.87$ gives an annual volatility

Part 1 Preparing the ingredients

Recipe 1.1 Stock the pantry

You will need

- Daily closing prices of about 250 exchange-traded funds and indices from Yahoo, with the *adjusted* close (dividends reinvested).
- Daily interest rates from FRED: the cash rate `DFF`, US Treasury yields at 2, 5, 10 and 30 years, the 10-year breakeven inflation rate, SOFR, the effective funds rate, the VIX level, and policy rates for the euro, yen and franc.
- Euro-area and Japanese government yield curves (ECB, Bank of Japan, Ministry of Finance), from the warehouse.
- The weekly Chicago Fed financial conditions index (NFCI), *as first published* and dated by the day it was published.
- For checking only: the Fama-French and AQR academic factors.

Method

1. Copy everything into one database. The warehouse tables are mirrored over a database link; Yahoo and FRED are downloaded fresh each night.
2. For each fund, keep the adjusted close. Mark each series as *total return* or *price only*.
3. Throw out, for factor building, every series that is price only: the `^`-prefixed index levels carry no dividends.
4. Throw out the `=F` futures series too: they are front-month prices glued together, with a fake jump at every roll.

Common mistake

Using a price index (no dividends) for an equity factor makes every equity beta too small by roughly the dividend yield. Using a stitched futures price for a commodity factor adds a jump every month that no investor ever earned. Both give factors that *look* fine and are wrong, which is worse than having no factor.

Done by: `pipeline/sync.py`, `pipeline/ingest_yahoo.py`, `pipeline/ingest_fred.py`.

Recipe 1.2 Lay out the calendar

All factors must live on the same list of days, or a "252-day window" would mean different things for different factors.

Method

1. Take every date on which any factor-input fund has a return.
2. Remove Saturdays and Sundays. (Crypto trades at weekends; left in, it added 1,229 days on which nothing else traded.)
3. Count, for each remaining date, how many factor-input funds reported. Find the largest count on any day.
4. Keep a date only if at least 30% of that largest count reported. This drops global holidays, when too few markets are open for a cross-asset factor to mean anything.

The dates left are the **trading calendar**. Everything below is placed on it.

Done by: `trading_calendar` in `pipeline/build_factors.py`.

Recipe 1.3 Turn prices into daily returns

Method

For each fund and each trading day:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

where P is the adjusted close. Use logs, not simple percentage changes: log returns add up over days, and converting currencies becomes a plain addition.

Worked example (illustrative numbers)

A fund closes at 100.00 and then at 100.80. Its log return is $\ln(100.80/100.00) = 0.00797$, i.e. 0.797%. (The simple return is 0.800%; for a daily move the two are almost the same.)

Recipe 1.4 Subtract the cash rate

A factor should measure the reward for taking a risk, not the interest you would have earned anyway. So every funded position has the day's cash rate taken off.

Method

$$r_t^{\text{excess}} = r_t - \frac{c_{t-1}/100}{252}$$

Use *yesterday's* cash rate: the overnight rate you earn on day t was set on day $t-1$. On a holiday with no rate printed, carry the last rate forward. (Here, carrying forward is honest: an unchanged policy rate is a fact, not an invention.)

Worked example (illustrative numbers)

The fund's log return is 0.797% and yesterday's `DFF` was 5.33%. A day's cash is $0.0533/252 = 0.0212\%$. The excess return is $0.797\% - 0.021\% = 0.776\%$.

Two kinds of factor skip this step because no money is invested in them: a *long-short* spread (the cash earned on the short pays for the long) and a *currency index* like the dollar index.

Recipe 1.5 Put everything in dollars

All factors are in US dollars. A London fund priced in pounds has to be converted.

Method

1. In logs, conversion is addition: $r_t^{\$} = r_t^{\text{local}} + r_t^{\text{FX}}$, where r^{FX} is the log change in dollars per pound.
2. Take the currency move *at the New York close*, the same moment every other price is taken. Get it from the currency's New York-listed trust (e.g. `FXB` for the pound, `FXY` for the yen), not from Yahoo's `=X` rate, which since August 2010 is a day late.
3. The trust earns a deposit rate minus a fee, which is not part of the exchange rate. Remove it: compute the trust's average daily drift over the spot rate across the past 252 days (ending yesterday) and subtract it from today's trust return.

$$s_t = r_t^{\text{trust}} - a_t, \qquad a_t = \frac{\ln \frac{P_{t-1}}{P_{t-253}} - \ln \frac{S_{t-1}}{S_{t-253}}}{252}$$

4. If the spot rate is missing for a few days, carry it at most 5 days. After that, the currency return is missing, and so is everything built on it.

Common mistake

Carrying an exchange rate forward without limit makes a currency that "stopped moving": every later change is exactly zero, and nothing downstream can tell that from a quiet market.

Done by: `fx_spot_return` and `Panels.in_base_ccy` in `pipeline/build_factors.py`.

Part 2 Cooking the factors

There are 51 factors: 40 core factors in nine blocks, and 11 sector factors that a single stock may add for its own sector. They are built with a small number of basic recipes. This part gives each basic recipe once; the table in Recipe 2.11 then says which recipe and which ingredients make each factor.

Every factor has a *level* in a hierarchy (0, 1, 2 or 3). Factors are cooked in level order, lowest first, because in Part 3 each factor is cleaned against factors of lower level, which must therefore already exist.

Recipe 2.1 The single-fund factor

Most factors are simply one fund's excess return. This is how global equity (`eq_global`, the MSCI ACWI fund), US investment-grade credit (`cr_ig`, `LQD`) or gold (`cm_gold`, `GLD`) are made.

Method

1. Take the fund's daily log return (Recipe 1.3).
2. Subtract cash (Recipe 1.4), unless the fund is a currency index.
3. If the fund is not priced in dollars, add the currency move (Recipe 1.5). Only the UK gilt fund `IGLT.L` needs this.

Recipe 2.2 The long-short spread

Size (`eq_size`) is small companies minus large companies: the Russell 2000 fund `IWM` minus the Russell 1000 fund `IWB`.

Method

$f_t = r_t^{\text{long}} - r_t^{\text{short}}$, using plain log returns. No cash is subtracted: the two cash legs cancel.

Recipe 2.3 The basket

Cyclical-minus-defensive (`eq_cyclical`) is the average of five cyclical sector funds (industrials, materials, energy, financials, consumer discretionary) minus the average of three defensive ones (staples, utilities, health care).

Method

$$f_t = \frac{1}{n_L} \sum_{\text{long}} r_t \; - \; \frac{1}{n_S} \sum_{\text{short}} r_t$$

Equal weights, plain log returns, no cash leg. The cross-asset risk-off factor (`liq_risk_off`) is made the same way: long Treasuries, the dollar and gold; short high-yield bonds and emerging-market stocks.

Recipe 2.4 A bond return from a yield

FRED gives yields, not prices. A yield going up is a bond price going down, so we convert each day's yield change into the return of an imaginary bond bought at par that day. This is a *synthetic par bond*.

You will need

The yield y of one maturity T (in years) for today and yesterday, as a decimal; yesterday's cash rate. Coupons are paid twice a year ($m = 2$).

Method

1. Compute the bond's **duration** at *yesterday's* yield. For a par bond:

$$D = \frac{1 - (1 + y_{t-1}/m)^{-mT}}{y_{t-1}}$$

   Duration is how much the price falls, in percent, for a 1-point rise in yield.

2. Compute its **convexity** C at yesterday's yield (the small curvature correction; the formula is in `core/transforms.py`).

3. Price the day:

$$r_t = \underbrace{-D\,\Delta y_t}_{\text{price move}} + \underbrace{\tfrac{1}{2}C\,(\Delta y_t)^2}_{\text{curvature}} + \underbrace{\frac{y_{t-1} - c_{t-1}}{252}}_{\text{carry above cash}}$$

Worked example (illustrative numbers)

The 10-year yield rises from 4.00% to 4.05%, so $\Delta y = 0.0005$. At 4% the duration is $D = 8.18$ and the convexity $C = 78.9$. Cash was 4.33%.

- price move: $-8.18 \times 0.0005 = -0.4088\%$
- curvature: $\frac{1}{2} \times 78.9 \times 0.0005^2 = +0.0010\%$
- carry: $(0.0400 - 0.0433)/252 = -0.0013\%$

The day's excess return is -0.409%.

Why it is done this way

Duration and convexity are taken at *yesterday's* yield. Taking them at today's yield would use the very move being priced to decide how it is priced.

Recipe 2.5 Yield-curve portfolios: level, slope, curvature

The curve factors (`rt_us_level`, `rt_us_slope`, `rt_us_curve`, and the euro and Japanese level factors) are fixed-weight portfolios of synthetic bonds.

Method

1. Make the synthetic bond return (Recipe 2.4) for each maturity needed. A missing yield may be carried at most 5 days (a local holiday), no further.

2. Divide each bond's return by its own duration (at yesterday's yield). This makes a "one-year-of-duration" return $u^{(T)}$, so a 2-year and a 30-year bond can be compared like for like.

3. Combine:

   - **Level** (2, 5, 10, 30 years): the average of the four u's.
   - **Slope** (10s minus 2s): $u^{(10)} - u^{(2)}$, a steepener.
   - **Curvature** (2, 5, 30 years): $2u^{(5)} - u^{(2)} - u^{(30)}$, long the middle, short the ends.

4. Multiply by 5 to scale back to the size of a 5-year-duration bond portfolio, a familiar magnitude.

Why it is done this way

Fixed weights rather than principal components: the three factors mean the same thing every day, their signs never flip, and no weight is fitted on future data. Principal components are still computed on the Covariance page as a check that the three span the same variance.

Breakeven inflation (`rt_breakeven`) uses the same bond recipe on the 10-year breakeven rate, but with the carry removed completely: a breakeven is the gap between two yields and pays no coupon of its own.

Recipe 2.6 Currency factors

Method

- **Dollar** (`fx_usd`): the dollar index's log return, no cash leg.
- **Yen** (`fx_jpy`): the New York-close spot move of the yen against the dollar (Recipe 1.5). Positive means a stronger yen, the usual "risk-off" direction.
- **Emerging currencies** (`fx_em`): the excess return of the `CEW` fund.
- **Carry** (`fx_carry`): for each of the euro, yen and franc,
  1. compute yesterday's rate gap: $g = (\text{foreign policy rate} - \text{US policy rate})/100/252$;
  2. hold the currency long if $g > 0$, short if $g < 0$;
  3. the leg's return is $\text{sign}(g) \times (\text{spot move} + g)$;

  then average the three legs.

Common mistake

The carry factor is an approximation. A real carry basket uses forward rates for ten currencies; there are no forward points in the data, and only three non-dollar policy rates. Treat a loading on it with suspicion.

Recipe 2.7 Factors from a rate or an index level: standardised changes

Some useful series are not prices at all: the SOFR-minus-funds-rate spread, the NFCI. Their units (basis points, index points) cannot be compared with returns.

Method

Daily series (SOFR minus EFFR, `liq_funding`):

1. Take the daily change of the spread.
2. Divide it by the standard deviation of the changes over the past 252 days (at least 60 needed). Now it is a "z-score": $+2$ means an unusually large widening.
3. Rescale the whole series by one constant so that it has 10% annual volatility, like a typical return factor: multiply by $\dfrac{0.10/\sqrt{252}}{\text{sd of the series}}$.

Why it is done this way

The constant in step 3 is a choice of units and nothing else. Multiplying a factor by k divides its beta by k and multiplies its volatility by k, so every predicted risk is unchanged. That is why it is safe to compute k from the whole sample.

Weekly series (NFCI, `liq_conditions`) get a different recipe, because filling a weekly number into every day of the week would invent six days of "information".

Method

1. Use each NFCI reading as first published, on the Wednesday it was published, not the Friday of the week it describes. If that Wednesday is a market holiday, move it to the next trading day.
2. On a publication day, the factor is the change since the last reading, divided by the standard deviation of the past 52 changes (at least 12).
3. On every other day, the factor is exactly **zero**.
4. Rescale to 10% annual volatility as above.

Common mistake

Dated by its reference Friday, the NFCI "knew" five days early what equities did that week: it correlated -0.33 with stock returns no one could yet have reacted to.

Recipe 2.8 Volatility factors

Method

- **Equity volatility** (`vol_equity`): the excess return of `VIXY`, a fund that holds VIX futures. A tradeable return, unlike the VIX level itself.
- **Variance risk premium** (`vol_variance_premium`): what a seller of one day's variance earns:

$$f_t = \frac{(\mathrm{VIX}_{t-1}/100)^2}{252} - r_{t,\mathrm{SPY}}^2$$

  i.e. yesterday's implied variance for one day, minus today's squared S&P 500 return. Then rescale to 10% volatility.
- **Rates volatility** (`vol_rates`): a "surprise" in the size of the Treasury fund `TLT`'s move:
  1. keep a running exponentially weighted average of TLT's squared returns with a 21-day half-life; its square root at yesterday's close is yesterday's volatility forecast s_{t-1};
  2. $f_t = |r_t|/s_{t-1} - 0.798$, where $0.798 = \sqrt{2/\pi}$ is the average size of a normal surprise, so the factor averages zero when the forecast is right;
  3. rescale to 10% volatility.

Worked example (illustrative numbers)

VIX closed yesterday at 20. One day's implied variance is $0.20^2/252 = 0.000159$. Today SPY moved 1%, so realised variance is $0.01^2 = 0.000100$. The short-variance

seller earned 0.000059 (before rescaling). On a 3% day they would have lost 0.000741: the payoff is small most days and sharply negative on bad ones.

Why it is done this way

`vol_rates` used to be the daily change in a 21-day rolling volatility. Half of that change was the squared return *leaving* the window, three weeks after it happened: an echo no security reacts to.

Recipe 2.9 Momentum strategies

The two alternative-risk-premia factors are trading rules run over 13 funds: US, developed and emerging stocks; long and medium Treasuries; commodities, gold, oil; high-yield and investment-grade credit; the dollar, the euro and the yen.

Method

Trend (`arp_trend`), for day t:

1. For each fund, add up its log returns over the past year but *skip the latest month*: the 231 days ending 22 days ago. The sign of that sum is the signal ($+1$ up-trend, -1 down-trend).
2. Compute its volatility over the 63 days ending yesterday.
3. Position = signal / volatility, so quiet assets get bigger positions.
4. The factor is $\sum(\text{position} \times \text{today's return}) / \sum|\text{position}|$.

Cross-sectional momentum (`arp_xs_momentum`):

1. Compute the same skip-a-month trailing return for every fund, and rank them.
2. Hold the top third long and the bottom third short, equally weighted within each.
3. The factor is the average return of the longs minus that of the shorts.

Recipe 2.10 Sector factors

Method

One factor per GICS sector, each the excess return of its SPDR sector fund (`XLK` for technology, `XLF` for financials, and so on). They are single-fund factors (Recipe 2.1). Two start late: real estate (`XLRE`) in October 2015 and communication services (`XLC`) in June 2018.

They are not part of the forty. Adding eleven more columns to every regression would cost more in noise than it explains. A stock may add the one factor of its own sector, and only if the agent finds it earns its place (Part 7).

Recipe 2.11 The full menu

The level column gives the cooking order; the last column says what each factor is cleaned against in Part 3.

Factor	Recipe	Made from	Lvl	Cleaned against
Equity market				
`eq_global`	single	ACWI	0	—
`eq_us`	single	IVV	1	`eq_global`
`eq_europe`	single	VGK	1	`eq_global`
`eq_japan`	single	EWJ	1	`eq_global`
`eq_em`	single	EEM	1	`eq_global`
`eq_size`	spread	IWM − IWB	1	`eq_global`
`eq_cyclical`	basket	5 cyclical − 3 defensive sector funds	1	`eq_global`
Equity style				
`sty_value`	single	VLUE	2	`eq_global, eq_us, eq_cyclical`
`sty_momentum`	single	MTUM	2	`eq_global, eq_us, eq_cyclical`
`sty_quality`	single	QUAL	2	`eq_global, eq_us, eq_cyclical`
`sty_lowvol`	single	USMV	2	`eq_global, eq_us, eq_cyclical`
Rates				
`rt_us_level`	curve	US 2/5/10/30y	0	—
`rt_us_slope`	curve	US 10y − 2y	0	—
`rt_us_curve`	curve	US 2/5/30y butterfly	0	—
`rt_ea_level`	curve	euro AAA 2/5/10/30y	0	`rt_us_level`
`rt_jp_level`	curve	JGB 2/5/10/30y	0	`rt_us_level`
`rt_uk`	single, in $	IGLT.L	0	`rt_us_level`
`rt_breakeven`	bond, no carry	10y breakeven	0	`rt_us_level`
Credit				
`cr_ig`	single	LQD	1	`rt_us_level, rt_us_slope`
`cr_hy`	single	HYG	1	`rt_us_level, rt_us_slope, eq_global`
`cr_loans`	single	BKLN	1	`rt_us_level, cr_hy`
`cr_em_hard`	single	EMB	1	`rt_us_level, rt_us_slope, cr_hy, eq_global`
`cr_em_local`	single	EMLC	2	`rt_us_level, cr_em_hard, fx_usd`
FX				
`fx_usd`	single, no cash	dollar index	0	—
`fx_jpy`	FX spot	yen via FXY	1	`fx_usd`
`fx_em`	single	CEW	1	`fx_usd`
`fx_carry`	carry	EUR, JPY, CHF rate gaps	1	`fx_usd, fx_jpy`
Commodities				
`cm_broad`	single	DBC	0	—
`cm_energy`	single	USO	1	`cm_broad`
`cm_gold`	single	GLD	1	`cm_broad`
`cm_industrial`	single	CPER	1	`cm_broad`
`cm_agri`	single	DBA	1	`cm_broad`
Volatility				
`vol_equity`	single	VIXY	1	`eq_global`
`vol_variance_premium`	variance payoff	VIX, SPY	1	`eq_global`
`vol_rates`	surprise	TLT	1	`rt_us_level`
Liquidity and stress				
`liq_funding`	std. change	SOFR − EFFR	0	—
`liq_conditions`	release event	NFCI	0	—
`liq_risk_off`	basket	TLT, UUP, GLD − HYG, EEM	2	`eq_global, rt_us_level, cr_hy`
Alternative risk premia				

Factor	Recipe	Made from	Lvl	Cleaned against
`arp_trend`	trend	13 funds	3	`eq_global, rt_us_level, cm_broad, fx_usd`
`arp_xs_momentum`	ranking	13 funds	3	`eq_global, arp_trend`
Sectors (role "sector", optional)				
`sec_* (11)`	single	XLK, XLF, XLV, XLI, XLY, XLP, XLE, XLB, XLU, XLRE, XLC	2	`eq_global, eq_us, eq_cyclical`

Done by: `pipeline/factor_defs.py` (the menu) and `pipeline/build_factors.py` (the cooking). Every formula is written out in full in `factor-formulas.md`.

Part 3 Separating the flavours

Recipe 3.1 Orthogonalise down the hierarchy

US stocks and global stocks move together almost perfectly (correlation 0.97). Put both raw into one regression and the betas come out as large numbers of opposite sign that mean nothing on their own. So each factor is cleaned of what the factors above it already explain. What is left answers a sharper question: `eq_us` becomes "what US stocks did *beyond* what global stocks did".

You will need

The raw factor f just cooked; the already-cleaned factors $X_1, \ldots, X_k$ it is to be cleaned against (last column of the menu).

Method

Go through the factors in level order. For each factor with something to clean against:

1. Wait until 252 days of history exist. Before that the cleaned factor is missing: that is the price of not using the future.
2. Every 21 trading days (about once a month), fit an ordinary least-squares regression of f on $X_1..X_k$ with an intercept, using only the **504 trading days before today** (about two years) and only days where every series is present:

$$f_s = a + b_1 X_{1,s} + \cdots + b_k X_{k,s} + e_s$$

3. For each of the next 21 days, the cleaned factor is what that fit does not explain:

$$f_t^{\perp} = f_t - a - b_1 X_{1,t} - \cdots - b_k X_{k,t}$$

4. Store both versions: the raw one (`ret_excess`) and the cleaned one (`ret_orth`).
5. When a later factor is cleaned against this one, use the *cleaned* version. The hierarchy compounds: credit is cleaned against cleaned rates, style against cleaned equity, and so on.

Factors with nothing in their last column are left as they are.

Worked example (illustrative numbers)

On the last refit, regressing `IVV`'s excess return on `eq_global` over the previous 504 days gave $a = 0.0001$ and $b = 0.95$. Today IVV returned 1.00% and global equity 0.80%. The cleaned US factor is $0.0100 - 0.0001 - 0.95 \times 0.0080 = 0.0023$, i.e. 0.23%: US stocks beat what the global market alone would have predicted by about a quarter of a percent.

Why it is done this way

Why a trailing two-year window, refitted monthly? Fitting once on the whole history would make the cleaned factors exactly uncorrelated, but would put 2026 information into 2008 values, flattering the model exactly where it is judged. A window that grows forever never forgets 2008, so its betas stay wrong for a decade. The trailing window leaves a small leftover correlation (-0.05 for emerging against global equity) and uses no future data. The leftover is not lost: the factor covariance in Part 5 measures it.

Common mistake

The cleaned factor is not what the factor *earned*. `eq_us` earns about 13% a year at 17% volatility; its cleaned version earns about -0.2% at 4%. That is why the terminal has a Factor Explorer (cleaned, what the model uses) and a separate Raw Explorer (raw, what the market did).

Done by: `core/orthogonalize.py` (`residualize_rolling`), called from `pipeline/build_factors.py`.

Recipe 3.2 Check that the style factors are what they say

The style factors are long-only funds, not the academic long-short portfolios. Check them against the published Fama-French and AQR factors, which arrive one to two months late and can never drive a daily model, but are good for checking.

Method

Correlate each cleaned factor with its academic counterpart over the days both exist.

Factor	Reference	Raw	Cleaned	Days
`eq_global`	Mkt-RF	0.963	0.963	4,551
`eq_size`	SMB	0.914	0.874	4,287
`sty_momentum`	Mom	0.207	0.638	3,006
`sty_value`	HML	0.194	0.499	3,006
`sty_lowvol`	BAB	−0.002	0.199	3,363
`sty_quality`	RMW	−0.145	0.143	2,943

Raw, the style funds barely resemble their targets, because market moves swamp the style. Cleaned, momentum and value become recognisable. Quality and low volatility remain weak: they are kept so the block is complete, not because they are good.

Recipe 3.3 Taste every factor: the stationarity battery

A factor must be a well-behaved return series: no trend in its level, no stale prices, no hidden break. Test this rather than assume it.

Method

For each factor, on its full history and on the last 252 days, run:

Test	What it catches	Effect
ADF and KPSS, read together	a level that should have been a change	can block
Zivot-Andrews	a one-off break that looks like a trend	warning
Variance ratio	stale or smoothed prices	warning
Ljung-Box	day-to-day autocorrelation	warning
Share of zero returns	a series that is not really trading	warning, or fail above 50%
ARCH-LM	calm and stormy spells	recorded only

Read ADF and KPSS as a pair, at 5%. ADF assumes "trending" and tries to reject it; KPSS assumes "steady" and tries to reject that. So:

- ADF rejects, KPSS does not ⇒ **stationary**, fine;
- KPSS rejects, ADF does not ⇒ **unit root**: the series blocks;
- both reject ⇒ probably a break or changing volatility: warning;
- neither rejects ⇒ inconclusive: warning.

Why it is done this way

Calm and stormy spells (ARCH effects) are *recorded but never block*. A series whose volatility clusters is still stationary, and almost every financial series clusters. What clustering tells you is to use robust standard errors and a fast-moving covariance, not to throw out the factor.

Done by: `core/stationarity.py`, `pipeline/run_diagnostics.py`.

Part 4 Fitting one security

Recipe 4.1 Prepare the security

Method

1. Pull its daily prices. Make log returns (Recipe 1.3), subtract cash (Recipe 1.4), and if it is not in dollars convert at the New York close (Recipe 1.5).
2. Run the stationarity battery on its returns.
3. Note its asset class (equity, Japanese equity, fixed income, commodity, FX), its GICS sector if it is a stock, and whether it closes before New York (a Tokyo stock does).

Recipe 4.2 Choose the factors

Method

Pick one of:

- **All** forty core factors.
- The **template** for the asset class: its own blocks plus a handful of cross-asset factors. For a US stock that is the equity and style blocks plus US level and slope, high yield, the dollar, broad commodities, energy, equity volatility and risk-off. Fixed income gets rates and credit plus global equity, the dollar, rates volatility, risk-off and financial conditions; and so on.
- Optionally, for a stock, add the factor of its own sector.

Choose also the **panel**: cleaned factors (the default, and the better forecaster) or raw factors (easier to explain: a beta of 1.02 to global equity means what it says).

Why it is done this way

Fewer factors means less noise. With forty factors on 252 days, the random error in the betas alone adds about $k/n = 40/252 \approx 16\%$ to the unexplained variance. A template keeps the factors that can plausibly matter.

Recipe 4.3 Run the rolling regression

You will need

The security's excess return y; the chosen factors' returns; a set of settings: window length (126 to 756 days), step (fixed at 21 days for the agent), estimator, weighting, Dimson lags, winsorising.

Method

Every 21 trading days (the *step*), on the date t that ends a window:

1. Take the last n days up to and including t (e.g. $n = 252$). Keep only factors with enough data in the window; drop the rest for this window.
2. Optionally **winsorise**: clip each series at its 1st and 99th percentile within the window, so one bad print cannot dominate.
3. Fit
$$y_s = \alpha + \beta_1 f_{1,s} + \cdots + \beta_k f_{k,s} + \varepsilon_s \qquad (s = t - n + 1, \ldots, t)$$
with one of
   - **OLS**: ordinary least squares, every day weighted equally;
   - **Huber**: like OLS but days with very large residuals get less weight (5 re-weighting passes, tuning constant 1.345), so one wild day pulls less;
   - **Ridge**: OLS plus a penalty λ that pulls betas towards zero, which steadies them when factors overlap.

   Optionally weight recent days more (EWMA weighting with a half-life of 42 to 252 days).

4. For a security that closes before New York, add **Dimson lags**: also regress on *yesterday's* (and the day before's) US-hours factors, because a Tokyo stock reacts to New York's move only the next morning. The total beta is the sum of the lags.
5. Record the betas, their full covariance V (robust Newey-West "HAC" errors), R^2, residual volatility, the condition number of the standardised factors, the largest variance inflation factor (VIF), and residual tests.
6. Record how much the beta vector changed since the last window (L1 distance and correlation, on the factors both windows share): a sudden jump is a drift alert.

The betas estimated at t are *in force* from t until the next window, 21 days later.

Common mistake

A window where few factors yet existed can be nearly singular: the factors are almost linear combinations of each other and the betas explode. Once that produced a 360% predicted volatility for Apple. So a window is flagged *ill-conditioned* when the condition number (on standardised factors) exceeds 200, or, on the cleaned panel only, when the largest VIF exceeds 100. Flagged windows are stored but never scored. (On the raw panel high VIFs are what the factor set is by design, so the VIF rule does not apply there.)

Done by: `core/regression.py`, `pipeline/run_estimation.py`.

Part 5 Forecasting the risk

The risk of a security is split in two: the part that comes from the factors (*systematic*), and the part that is its own (*specific*).

$$\sigma^2_{\text{total}} = \underbrace{\beta^\top \Sigma_F \beta}_{\text{systematic}} + \underbrace{\sigma^2_\varepsilon}_{\text{specific}}$$

Recipe 5.1 Estimate the factor covariance

You will need

Daily returns of the factors the betas refer to, *from the same panel* (cleaned betas need the covariance of cleaned factors), for a trailing window (252, 504 or 756 days) ending on the forecast date.

Method

Choose one estimator:

- **Sample**: the ordinary covariance, every day equal.
- **EWMA**: recent days count more; a day j days old has weight $0.5^{j/h}$ for half-life h (5 to 126 days). Short half-lives follow volatility quickly.
- **Ledoit-Wolf**: the sample covariance pulled part of the way towards a simple target in which every pair of factors has the same correlation. Steadier, especially when there are many factors and few days.

- **Blend**: $a \times$EWMA $+ (1-a) \times$Ledoit-Wolf. The original model used $a = 0.6$ with a 60-day half-life.

Annualise (times 252). Check that the matrix is valid (no negative eigenvalues). If it is not, set the negative eigenvalues to a tiny positive number and then rescale so each factor's volatility is exactly what it was before.

Recipe 5.2 Systematic risk, with the noise taken out

Method

1. Take the betas in force on the forecast date and compute $\beta^\top \Sigma_F \beta$.
2. Optionally **debias**: subtract $\mathrm{tr}(\Sigma_F V)$, where V is the betas' covariance from the regression. Estimated betas are noisy, and noise always *adds* to $\beta^\top \Sigma \beta$ on average. This term is that addition.

Recipe 5.3 Specific risk

Method

Choose one:

- **In-sample**: the residual volatility of the regression window.
- **Out-of-sample**: each day, take the security's return minus what the betas in force predicted. Square it, subtract the beta-noise part $f^\top V f$ (so it is not counted twice), and keep an exponentially weighted average (half-life 21, 63 or 126 days).

Never let specific volatility fall below **2% a year**. The floor is the model admitting it cannot see every risk.

Recipe 5.4 Put the forecast together

Method

1. Total variance = systematic + specific. Its square root is the annual volatility forecast; divide by $\sqrt{252}$ for a day.
2. **Refresh**: either keep the forecast fixed until the next regression window (monthly), or recompute the covariance and specific risk every day while holding the betas. Daily refresh follows volatility far better.
3. **Value at Risk**: the 95% one-day VaR is $z_{0.95} \times$ daily volatility, with $z = 1.645$ for a normal distribution (2.326 at 99%). Or use a Student t whose fatness is fitted to the security's own past returns divided by their own EWMA volatility, which gives larger tails.

Worked example (illustrative numbers)

A stock has betas 1.1 on global equity and 0.3 on a style factor. Global equity has 16% volatility, the style factor 8%, correlation 0.2. Specific volatility is 20%.

- systematic: $1.1^2(0.16^2) + 0.3^2(0.08^2) + 2(1.1)(0.3)(0.2)(0.16)(0.08) = 0.0332$

- specific: $0.20^2 = 0.0400$
- total: 0.0732, so volatility $= \sqrt{0.0732} = 27.1\%$ a year, $27.1/15.87 = 1.70\%$ a day
- 95% one-day VaR (normal) $= 1.645 \times 1.70\% = 2.80\%$

Systematic risk is 45% of the total; the rest is the stock's own.

Recipe 5.5 The one-month forecast

A month's risk is not simply a day's risk times $\sqrt{21}$, for two reasons.

Markets close at different times. The euro and Japanese curves, a London fund and the FX fixes close before New York, so one day's shock lands in two days' returns. A daily covariance cannot see this; a 21-day return contains it.

Method

Add the lead-lag term: with Γ_l the covariance of factors today with factors l days earlier, the h-day variance per day is

$$\Gamma_0 + \sum_{l=1}^{L} \left(1 - \frac{l}{h}\right)\left(\Gamma_l + \Gamma_l^{\top}\right)$$

with $L = 0$, 1, 2, 3 or 5 lags (zero unless the evidence is clear). For a Dimson model, use the summed beta here.

Volatility reverts. After a shock it falls; after a calm spell it rises. A forecast made today should drift towards the long-run level over the month.

Method

With half-life H (126, 63 or 21 days, or none), $\varphi = 0.5^{1/H}$ and horizon h:

$$\text{month forecast} = LR + g\,(\text{today} - LR), \qquad g = \frac{1 - \varphi^h}{h(1 - \varphi)}$$

LR is the long-run level: the Ledoit-Wolf covariance of the same window, and the in-sample specific variance. For a day ahead $g = 1$, so the daily forecast is untouched.

Worked example (illustrative numbers)

With $H = 63$ and $h = 21$: $\varphi = 0.98906$ and $g = 0.898$. If today's variance is twice the long-run level, the month forecast is $LR + 0.898\,LR = 1.90 \times LR$: the shock is expected to fade a little within the month.

Done by: `core/covariance.py`, `pipeline/run_risk.py`.

Part 6 Tasting the dish: scoring the forecast

A forecast is only worth something if it is right about the future. So every forecast dated t is compared with what happened on days $t+1$ to $t+h$. The betas come from a window ending at or before t, the covariance from returns up to t: the gap is built in, not a convention.

Method

Collect all forecasts with their outcomes, excluding ill-conditioned windows, and compute:

1. **Bias statistic**. Divide each return by the volatility forecast in force for it: $z = r/\hat{\sigma}$. The standard deviation of the z's should be 1. Above 1: risk was under-forecast. Below 1: over-forecast. With n forecasts, anything within $1 \pm \sqrt{2/n}$ is acceptable.
2. **Mincer-Zarnowitz**. Regress realised variance on forecast variance: realised $= a + b \times$forecast. A good forecast has $a = 0$, $b = 1$; test both together.
3. **Kupiec**. Count the days the loss exceeded the VaR. At 95% you expect 5% of days. Is the count too far from that?
4. **Christoffersen**. Are the breaches independent, or do they come in bunches? A model can have exactly the right count, all in one week.
5. **Against a naive forecast**. Score the model, the security's own EWMA volatility and a one-factor market model with the QLIKE loss,

$$\text{QLIKE} = \frac{\text{realised}^2}{\text{forecast}^2} - \ln \frac{\text{realised}^2}{\text{forecast}^2} - 1,$$

which is zero for a perfect forecast and ranks variance forecasts fairly even when "realised" is noisy. Compare with a Diebold-Mariano test. A model that is significantly *worse* than the security's own past volatility has not earned its factors.

Common mistake

A Mincer-Zarnowitz slope below 1 is partly unavoidable. Realised volatility over the next 21 days mixes calm and stormy days, while the forecast refers to one moment. In a simulation with a *perfect* forecast, regimes lasting 21 days gave a slope of 0.50. So a low slope is not by itself proof the model over-reacts.

Done by: `core/risk.py`.

Part 7 Choosing the recipe: the model agent

Parts 4 and 5 left many choices open: which factors, which window, which estimator, which covariance, how often to refresh. None is right for every security. The agent tries many combinations for one security and picks one. A language model (DeepSeek) suggests what to try next; *every number is computed by the code*, and the language model never fits, scores, picks the winner or writes the verdict.

Recipe 7.1 Lock away the test data

Method

1. Lay a monthly grid of forecast dates (every 21 trading days) over the security's history.
2. The last 36 grid dates (or 30% of the grid if that is fewer, never fewer than 24) are the **test span**. Put them in a locked drawer.

3. Everything before is the **validation span**, less an *embargo*: drop validation dates whose 21-day outcome would reach into the test span.
4. During the search, the code literally hands the scoring routine a copy of the data that ends the day before the test span. Nothing can peek.

Why it is done this way

Try forty recipes and keep the one that scored best, and you have found the luckiest, not the best. Its score is flattered by the search. The only honest score is one on data the search never touched, taken once. A test runs the whole agent twice, once with every test-span return multiplied by fifty, and checks the choice is identical.

Recipe 7.2 Gather the candidates

Each candidate is one full recipe: the fitting half (panel, factors, window, weighting, estimator, Dimson lags, winsorising, sector) and the risk half (covariance method, half-life, blend weight, window; specific risk; debiasing; refresh; VaR distribution).

Method

1. **Seed** with fixed starting points: the original model (cleaned panel, all factors, 252-day OLS, 60/40 blend, held for a month); the raw panel; ridge; a longer decayed window; a fast, daily-refreshed, debiased forecast with t-VaR; the asset-class template; one Dimson lag for an early-closing security; the template with its own sector for a stock.
2. **Pool**: add up to eight recipes that won or nearly won in earlier runs, for this security or similar ones. Only the recipes travel, never their scores.
3. **Score** each candidate on the validation span: run Parts 4 and 5, and compute the daily QLIKE of the forecast in force each day (scored against that day's squared return).
4. **Propose more**. The language model sees the profile, every candidate's validation numbers and the critic's hints, and replies with up to six new candidates, or "stop". The *critic* is a set of plain rules that read the leader's diagnostics: "loses to own EWMA, so the forecast is too slow; try a shorter half-life"; "99% breaches far too many while 95% is fine, so the tails are too thin; try t-VaR". If the language model fails twice, the critic proposes instead.
5. Repeat steps 3 and 4 for at most five rounds, forty candidates or thirty minutes.

Recipe 7.3 Pick the winner

Method

1. **Model confidence set**. Find every candidate that cannot be told apart from the best at 90% confidence (Hansen-Lunde-Nason, with a block bootstrap of the daily loss). The rest are significantly worse and drop out.
2. **Calibration screen**. Within the set, keep only candidates whose validation forecasts were calibrated: bias statistic within $1 \pm \sqrt{2/n}$ and Kupiec not rejected at 95% or

99%. (If none pass, keep the set.)

3. **Simplest wins**. Among those left, pick the one with the fewest effective parameters (the number of factors; for ridge, fewer because the betas are shrunk). Break ties on the loss. The stock's own sector factor is not counted as a parameter: its value lies in what stocks of one sector share, which one stock's loss cannot see.
4. **Fix the month forecast**. With the winner chosen, try 0, 1, 2, 3 and 5 lead-lag lags, then mean reversion of none, 126, 63 and 21 days, on the validation span's 21-day returns. Keep the simplest whose loss is within one standard error of the best.

Recipe 7.4 Open the drawer once: the verdict

Method

Run the chosen recipe on the test span and check each gate:

Gate	Rule
Correctness: any failure is a fail	
test sample	at least 24 test forecasts, else "insufficient data"
forecasts issued	a forecast on at least 95% of test dates
conditioning	at most 10% of test windows ill-conditioned
stationary	the security's returns do not fail the battery
Value: failure on the monthly loss is a fail	
beats naive	not significantly worse than own EWMA on monthly QLIKE ($p < 0.05$)
…day by day	the same on the daily loss; losing is a caveat
Calibration: each failure is a caveat	
bias	21-day bias statistic within $1 \pm \sqrt{2/n}$
Mincer-Zarnowitz	$a = 0$, $b = 1$ not rejected
VaR count	Kupiec not rejected at 95% and at 99%
VaR clustering	Christoffersen not rejected at 95%
circularity	the security is not itself an input to one of its factors

No caveats: **pass**. One or two: **pass with caveats**. More: **fail**.

The chosen recipe, its betas, forecasts and both backtests (full history and test span only) are stored, so every page of the terminal reads it like any other model. Forecasts dated before the test span are marked *in selection sample*: they include the dates the recipe was chosen on, and are not evidence for it.

Done by: `agent/graph.py` (the workflow), `agent/space.py`, `agent/evaluate.py`, `agent/critic.py`, `agent/llm.py`, `core/selection.py`.

Part 8 Serving a portfolio

You will need

Weights w_i for each holding; each holding's betas β_i (a 252-day OLS refitted every 21 days on the cleaned factors, its asset-class template, plus its own sector factor for a stock or a sector fund), beta covariances V_i and specific variances.

Method

1. Add up the exposures: $x = \sum_i w_i \beta_i$ (the portfolio's beta on each factor).
2. Estimate the factor covariance as a mix of three speeds: 80% a 5-day EWMA, 10% a 63-day EWMA, 10% a 252-day EWMA. It reacts within days without collapsing in one quiet week.
3. Next-day variance:

$$\hat{\sigma}_p^2 = x^\top \Sigma_F x \; - \; \sum_i w_i^2 \, \mathrm{tr}(\Sigma_F V_i) \; + \; \sum_i w_i^2 \hat{\sigma}_{\varepsilon,i}^2$$

   Specific risks add with squared weights: they are assumed unrelated across holdings, which is why stocks need their sector factor (it moves the shared sector part out of "specific").
4. For the month, revert towards the 252-day level with a 63-day half-life (Recipe 5.5).
5. Split the risk by **Euler contributions**: each holding's (or factor block's) share is its weight times how much the total would change per unit more of it. The shares add up to 100%.
6. Against a benchmark, run the same recipe on the *active* weights (portfolio minus benchmark) to get the tracking error.

Common mistake

A fund that is itself an input to a factor (say `HYG`, which *is* the high-yield factor) has loadings that just restate the factor's recipe. Its risk still adds up correctly, but its betas explain nothing. The terminal flags it.

Done by: `pipeline/portfolio_risk.py`, `core/portfolio.py`, `pipeline/run_portfolio.py` (the 54-stock validation).

Appendix A Letting the machine cook

Everything above, in the order a fresh install runs it:

```
python -m backend.pipeline.apply_schema # create the database
python -m backend.pipeline.sync --full # Part 1:  warehouse data
python -m backend.pipeline.ingest_yahoo --full# Part 1:  prices
python -m backend.pipeline.ingest_fred --full # Part 1:  rates
python -m backend.pipeline.build_factors # Parts 2-3:  factors, cleaned
python -m backend.pipeline.run_diagnostics # Part 3:  stationarity battery
python -m backend.pipeline.sync --security US:AAPL
python -m backend.agent.run --instrument US:AAPL # Parts 4-7
python -m backend.pipeline.run_portfolio # Part 8:  54-stock check
```

Or, for a hand-set recipe instead of the agent's:

```
python -m backend.pipeline.run_estimation --instrument US:AAPL --window 252 --step 1m
python -m backend.pipeline.run_risk --instrument US:AAPL --horizon 21
```

Appendix B Kitchen glossary

Beta (loading)	how much a security moves per unit move of a factor
Cleaned factor	a factor with what the higher-level factors explain removed (orthogonalised)
Condition number	how close the factors in a window are to repeating each other; large means unstable betas
Covariance	how much two series move together; on the diagonal, each one's variance
Dimson lag	regressing on yesterday's factors too, for a market that closes earlier
EWMA	exponentially weighted moving average: recent days count more, halving every half-life
Excess return	return minus what cash would have paid
Half-life	the age at which a day's weight has halved
HAC errors	standard errors that allow for autocorrelation and changing volatility
Ledoit-Wolf	a covariance pulled towards a simple target to reduce noise
QLIKE	a score for a variance forecast; zero is perfect, lower is better
Specific risk	the part of a security's risk the factors do not explain
Stationary	a series whose behaviour does not drift over time
Test span	the last stretch of history, kept unseen until the recipe is chosen
VaR	Value at Risk: a loss exceeded on only 5% (or 1%) of days
VIF	variance inflation factor: how much one factor is explained by the others
Winsorise	clip extreme values at chosen percentiles